

Supervised learning

INF-GRAM — Machine Learning for Healthcare

Oskar Laverny

September 16, 2026

Session roadmap



What we will study today

We focus on the most common predictive setting in machine learning:

- a set of predictors (X);
- a target variable (y);
- a model that learns from labelled examples.

Learning objectives

By the end of the lecture, you should be able to:

- distinguish classification and regression;
- explain why simple baselines matter;
- understand the intuition behind logistic regression and SVMs;
- explain why scaling and pipelines are important;
- describe a safe workflow for model selection.

What is a supervised-learning problem?

Inputs, labels and prediction

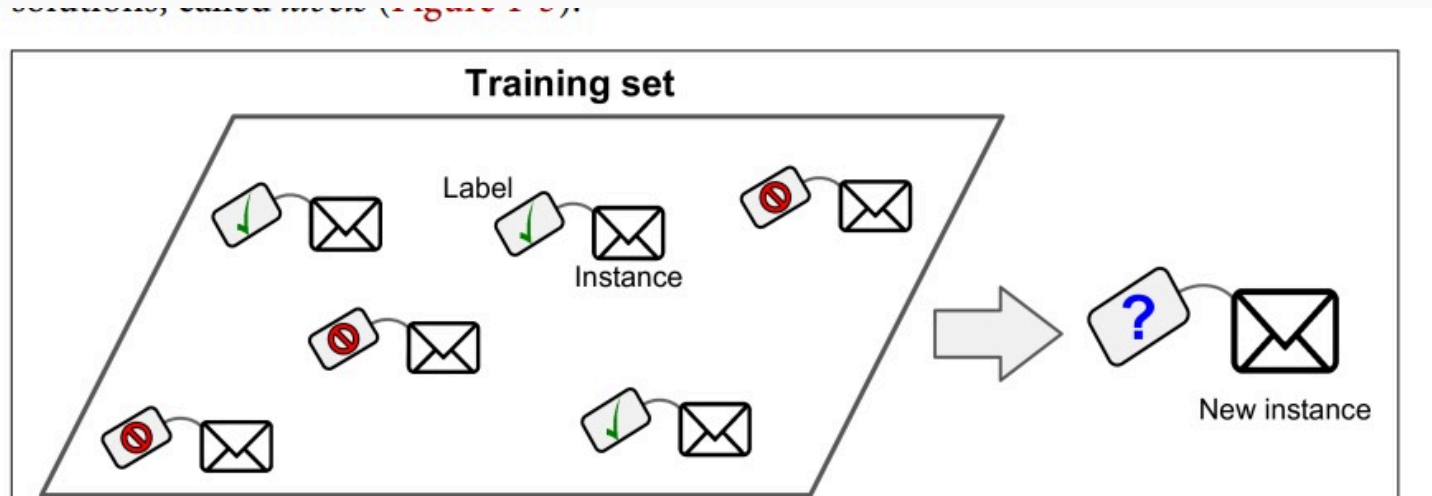
In supervised learning we observe:

$$[X = , y = .]$$

The goal is to learn a function (f) such that:

$$[y = f(X).]$$

Labelled examples



The training set contains past examples for which the correct answer is already known.

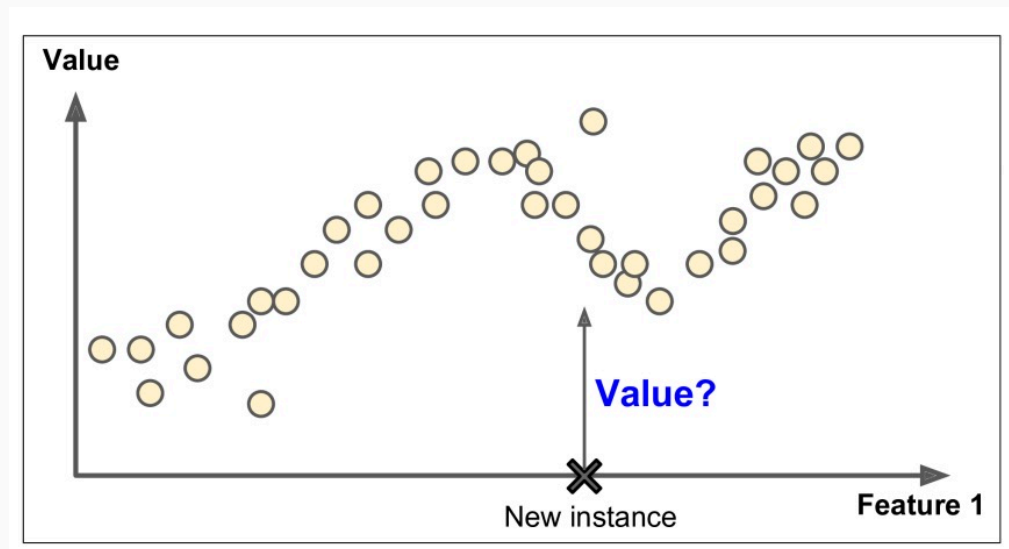
Two major task types

Classification

- predict a category;
- examples: positive / negative, responder / non-responder.

Regression

- predict a continuous number;
- examples: length of stay, risk score, biomarker value.



A clean modelling workflow

A good supervised-learning workflow is:

1. split into train and test sets;
2. choose a baseline;
3. preprocess data correctly;
4. train and tune candidate models;
5. evaluate once on the untouched test set.

Baselines first, complexity second



Why start simple?

Before trying sophisticated methods, compare with:

- a dummy classifier;
- logistic regression for classification;
- linear regression for continuous outcomes.

A complex model should **earn** its extra complexity.

Why logistic regression is still important

Logistic regression remains a strong reference model because it is:

- fast;
- stable;
- interpretable;
- naturally probabilistic;
- often surprisingly competitive.

Logistic regression for binary classification

$$\ln \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p.$$

The important practical message is not the derivation itself:

- the model estimates a probability;
- coefficients describe how predictors are associated with the log-odds;
- regularization may still be needed in high dimension.

Preprocessing and scaling

Why scaling matters

Many algorithms are sensitive to the numerical scale of the variables.

If one feature ranges from 0 to 10,000 and another from 0 to 1, then the first one may dominate distances or margins.

A common transformation is:

$$[z = .]$$

The golden rule: split first, preprocess second

To avoid leakage:

- fit the scaler on the training data only;
- reuse that fitted scaler on validation and test data;
- put preprocessing and model fitting inside a single `Pipeline`.

This becomes crucial when cross-validation is used.

Support vector machines

Intuition: many boundaries may separate the classes

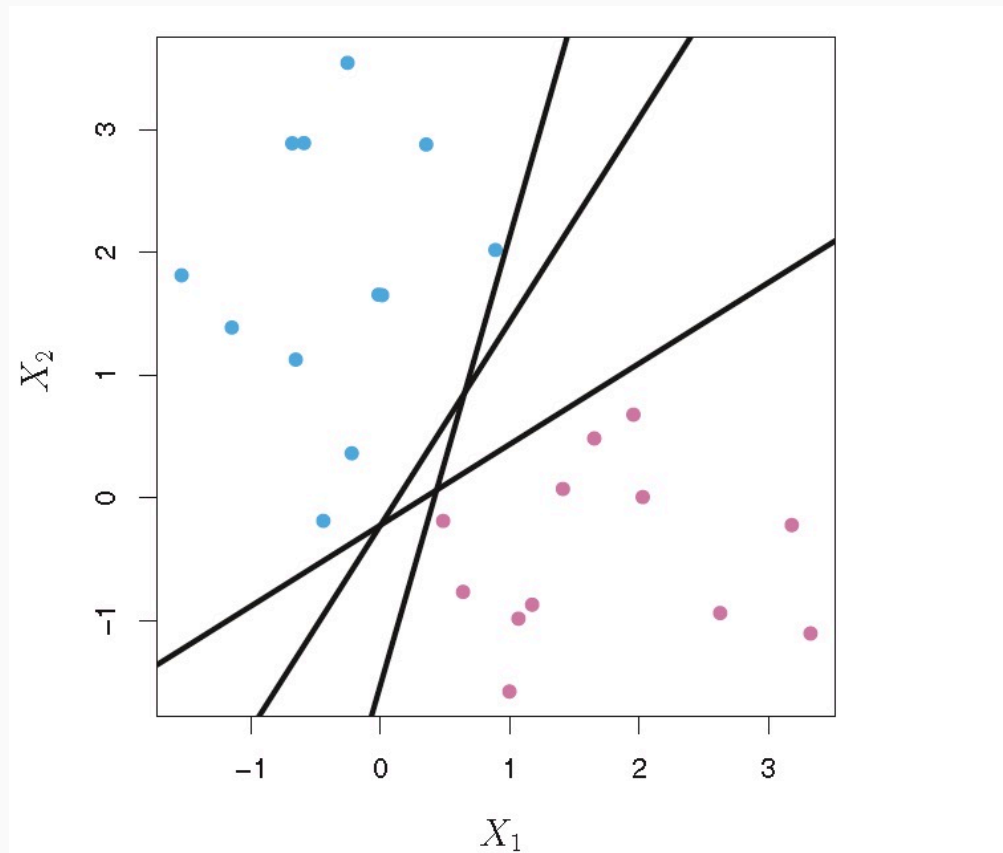
If the classes are separable, many linear boundaries may classify the training set correctly.

The question is therefore not only:

Can we separate the classes?

but also:

Which separating boundary should we prefer?

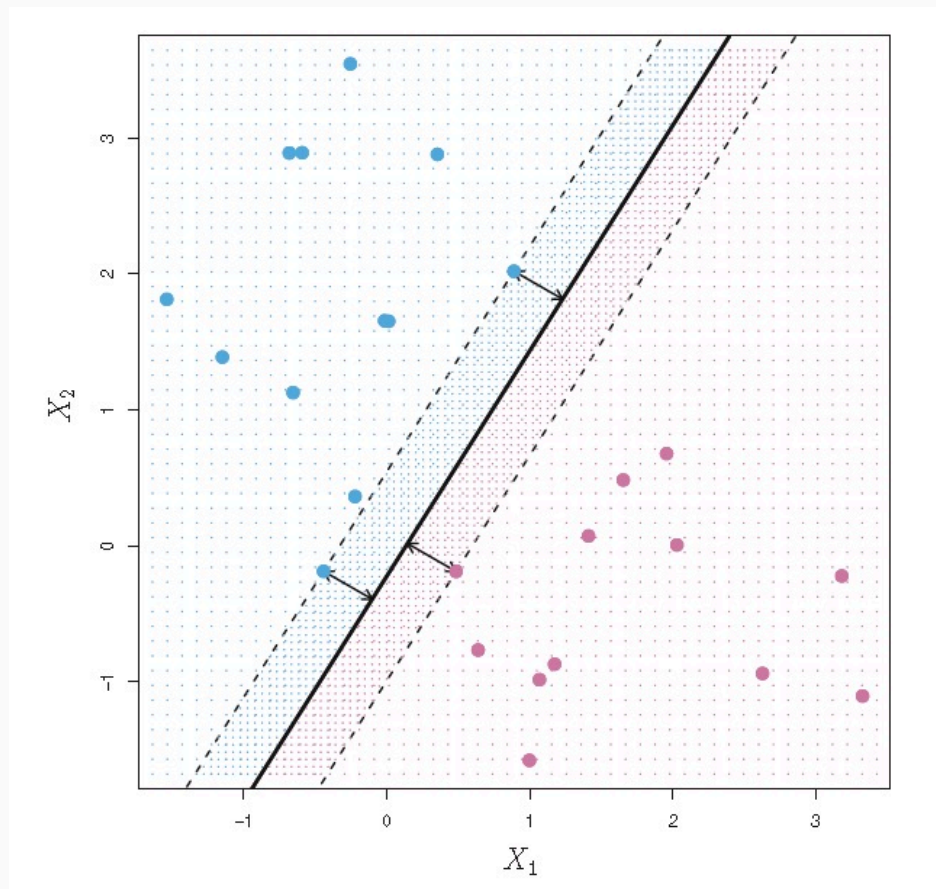


SVM chooses a large-margin boundary

The support vector machine prefers a boundary with a **large margin**.

Only a subset of observations — the **support vectors** — determine the final solution.

This tends to improve robustness.

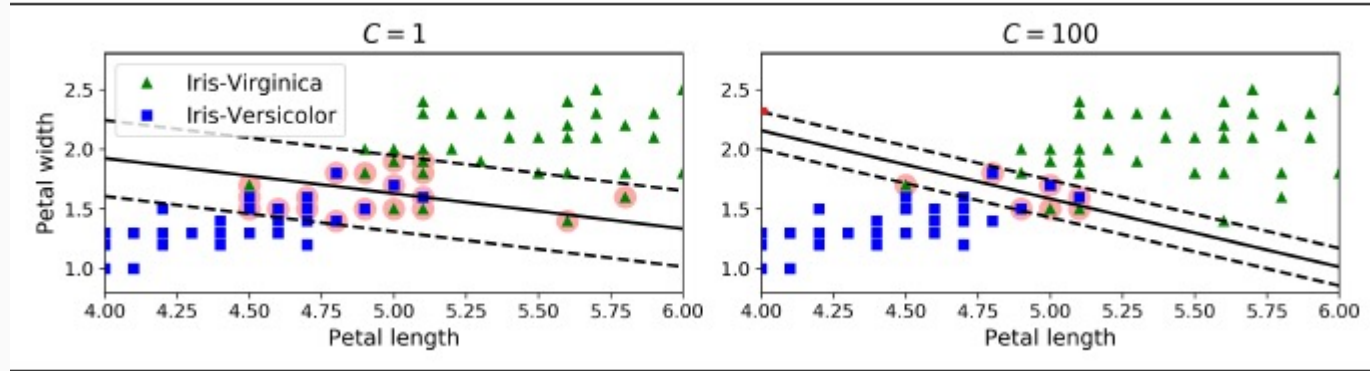


C controls the trade-off between:

- a larger margin;
- a stronger penalty on training errors.

Small C means stronger regularization. Large C allows the model to fit the training data more closely.

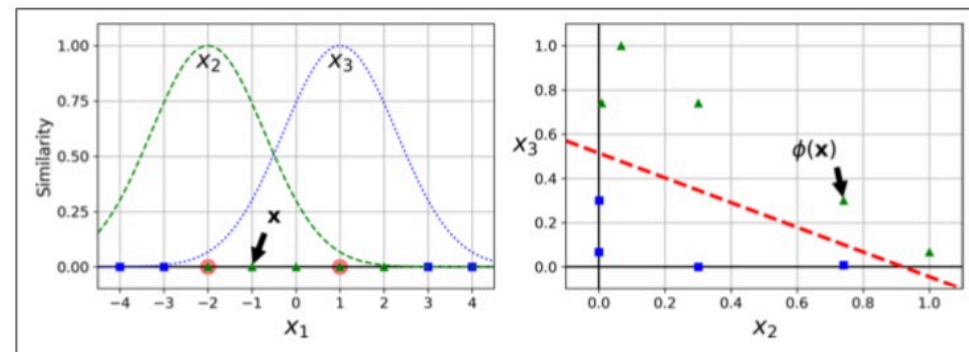
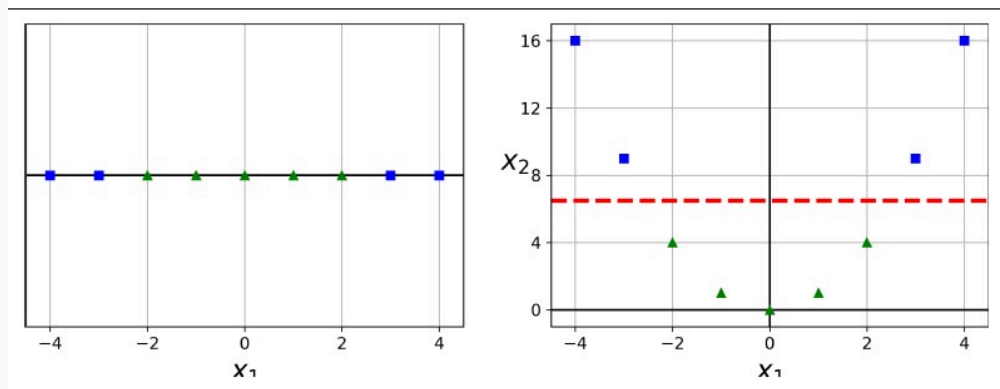
Visual effect of C



When a linear boundary is not enough

Some classification problems are not linearly separable in the original feature space.

A non-linear transformation may reveal a simpler separation.



The kernel trick in one sentence

A kernel method behaves **as if** we had mapped the data to a richer feature space, without explicitly constructing all transformed variables.

This is what makes SVMs flexible and powerful.

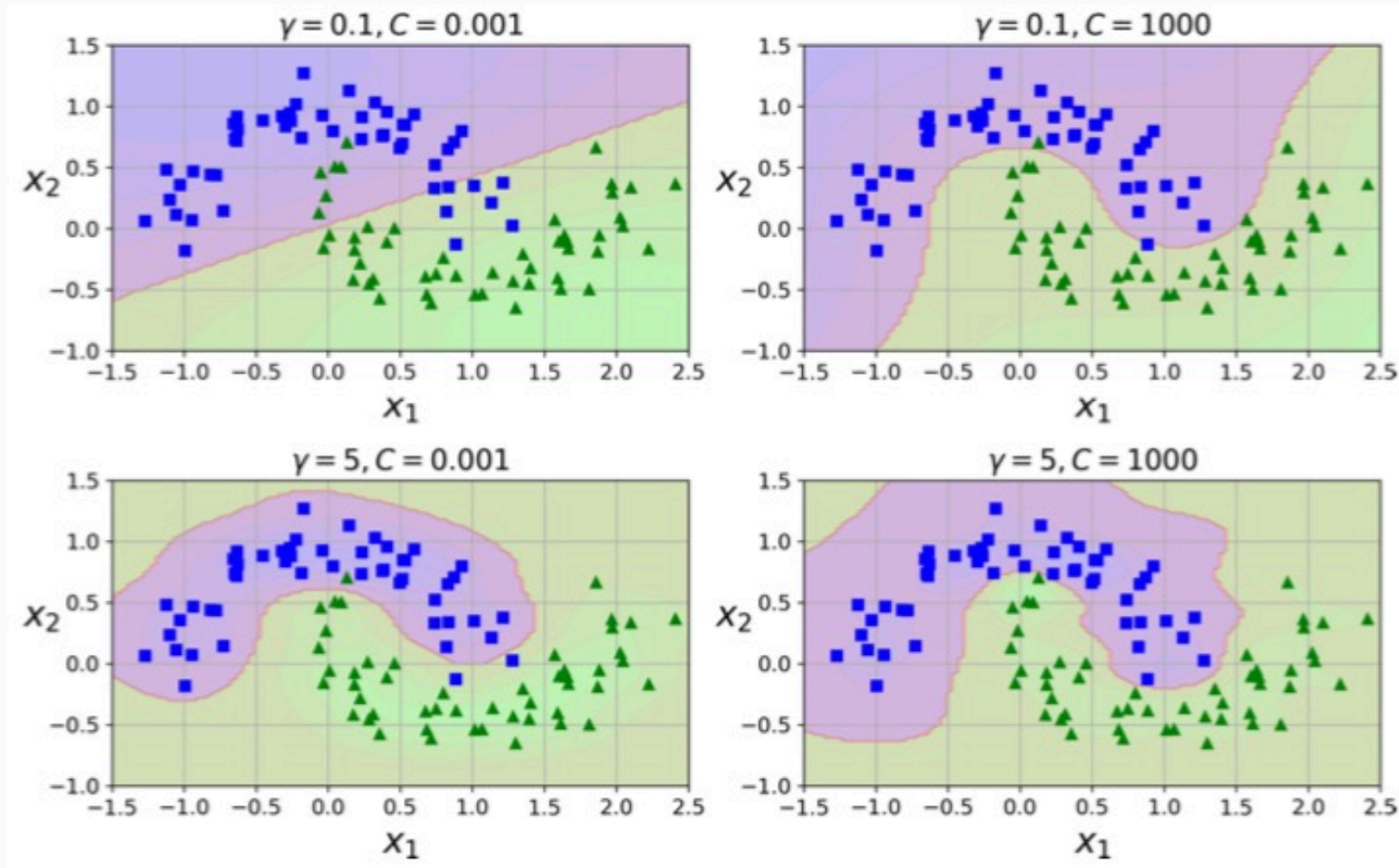
Tuning an RBF SVM

The two main hyperparameters are:

- **C**: how much we penalize training errors;
- **gamma**: how local / wiggly the boundary becomes.

Large **C** and large **gamma** can overfit quickly.

Effect of C and γ



Model selection without leakage

A safe workflow

1. Split into train and test.
2. Build a **Pipeline** with preprocessing + model.
3. Tune hyperparameters by cross-validation on the training set only.
4. Refit the selected pipeline.
5. Evaluate once on the test set.

What to avoid

Do **not**:

- scale before splitting;
- tune directly on the test set;
- compare many models and keep peeking at the final test score.

Take-home messages

What to remember from this lecture

- Supervised learning uses labelled examples to learn a prediction rule.
- Always start with a strong baseline before trying complex models.
- Scaling and pipelines are central to a correct workflow.
- SVMs are built around the ideas of margin, regularization and kernels.
- Hyperparameter tuning must never contaminate the final evaluation.